

REVIEW 0

Problem · Motivation · Objectives

Smart Inventory & Warehouse Management System

Team Members

S Svara Haasini – 2520090170

B Sudharshini – 2520090227

Samyuktha Kandukuri – 2520090157

Database Systems Engineering & Distributed Backend Development (25CS1302E)

Under the Guidance of: Course Faculty Guide

Agenda

- Abstract
- Problem Statement
- Motivation
- Objectives

Abstract

- A database-driven application to centrally manage products, stock levels, suppliers, warehouses, and inventory transactions.
- Replaces error-prone manual record keeping with a MySQL database exposed through a Flask web interface.
- Automatically updates stock on every purchase, transfer, or dispatch, and flags low-stock items for timely restocking.
- Uses SQL joins, aggregate functions, views, and stored procedures for accurate, redundancy-free data management.
- **DBMS**
- Core technology: relational database design & SQL

Problem Statement

- **Existing problem:** small and mid-sized warehouses still track stock with spreadsheets, paper registers, or basic POS tools with no database backing.
- **Who is affected:** warehouse staff, suppliers, and managers who need accurate, real-time stock visibility across locations.
- **Why it matters:** manual methods are slow, error-prone, and make searching, reporting, and restocking decisions unreliable.
- **Key limitation:** no structured relationships between products, suppliers, and warehouses, leading to redundant and inconsistent data.
- **Problem definition:** design a relational database system that centralizes inventory data and automates stock updates and low-stock alerts.

Motivation

- **Why this project:** inventory mismanagement is one of the most common and costly problems for growing businesses.
- **Real-world importance:** accurate stock data directly affects order fulfilment, supplier trust, and warehouse efficiency.
- **Need for automation:** manual restocking checks and duplicate data entry waste staff time and introduce errors.
- **Expected benefit:** a single reliable source of truth for products, stock, suppliers, and transactions.
- **Academic relevance:** hands-on application of core DBMS concepts — schema design, normalization, joins, and stored procedures.

Objectives

- Design a normalized relational schema linking Products, Categories, Suppliers, Warehouses, Stock, and Transactions.
- Build a Flask + MySQL backend to add, update, search, and delete inventory records without writing raw SQL.
- Automatically update stock levels on every purchase, transfer, or dispatch transaction.
- Flag products that fall below a defined reorder threshold for timely restocking.
- Enable fast reporting using SQL joins, aggregate functions, views, and stored procedures.
- Core objectives guiding the system design